

Activités d'Intégration de Compétences —

Partie 2

Objectif général

Enrichir le projet Spring Boot de la Partie 1 en ajoutant de la logique métier testée, une interface web légère, une image Docker publiée, un déploiement Kubernetes, et une architecture de micro service plus complète avec cache, gestion des erreurs, documentation et traçabilité Jira.

Q1 — Créer une branche Git `version-2`

Indication : Le travail de cette partie doit être isolé dans une branche dédiée. Créez la branche à partir de `main` (ou `master`) et basculez dessus avant de commencer.

```
git checkout -b version-2
```

Q2 — Ajouter une méthode `age()` à l'entité `Etudiant`

Indication : La méthode doit calculer dynamiquement l'âge de l'étudiant à partir de son `dateNaissance` (de type `LocalDate`) et de la date du jour. Utilisez `java.time.Period` ou `ChronoUnit.YEARS`. Avec Lombok, pensez à annoter correctement la classe pour éviter les conflits avec les getters générés.

```
public int age() {  
    return Period.between(this.dateNaissance, LocalDate.now()).getYears();  
}
```

Q3 — Tester la méthode `age()` avec la syntaxe Gherkin (BDD)

Indication : Utilisez **Cucumber** avec JUnit 5 pour écrire un test de style Behaviour-Driven Development. Créez un fichier `.feature` décrivant le scénario en langage naturel (Given / When / Then), puis implémentez les step definitions correspondantes en Java. L'objectif est de vérifier que `age()` retourne la valeur attendue pour une date de naissance donnée.

Exemple de scénario Gherkin :

Feature: Calcul de l'âge d'un étudiant

Scenario: Étudiant né il y a 22 ans

Given un étudiant avec la date de naissance "2002-04-07"

When on calcule son âge

Then l'âge retourné doit être 23

Q4 — Ajouter une page `index.html` avec un appel Fetch JavaScript

Indication : Placez le fichier `index.html` dans le dossier `src/main/resources/static/` de votre projet Spring Boot. Spring Boot sert automatiquement les fichiers statiques depuis ce répertoire. La page doit effectuer un `fetch('/api/etudiants')` et afficher dynamiquement la liste dans le DOM (table HTML ou liste). Aucun framework JavaScript n'est requis.

```
<!-- Exemple minimal -->
<script>
  fetch('/api/etudiants')
    .then(res => res.json())
    .then(data => {
      // Construire et insérer les éléments dans le DOM
    });
</script>
```

Q5 — Créer et publier l'image Docker du micro service sur Docker Hub

Indication : Construisez l'image Docker de votre API Spring Boot à partir du `Dockerfile` de la Partie 1, taguez-la avec votre nom d'utilisateur Docker Hub, puis poussez-la. Cette étape simule la publication d'un artefact prêt à déployer.

```
docker build -t <votre-username>/etudiant-service:1.0 .
docker push <votre-username>/etudiant-service:1.0
```

Q6 — Créer les manifests Kubernetes pour déployer l'application sur K3S

Indication : Rédigez deux fichiers YAML Kubernetes : un `Deployment` + `Service` pour le micro service étudiant, et un `Deployment` + `Service` pour PostgreSQL. Pour cette étape, **aucune persistance n'est requise** (pas de PVC). Déployez sur une VM locale équipée de **K3S** (distribution Kubernetes légère). Vérifiez l'accès au service via `kubectl port-forward` ou un NodePort.

Fichiers attendus :

- k8s/etudiant-deployment.yaml
- k8s/postgres-deployment.yaml

Q7 — Ajouter l'entité Département et mettre à jour le schéma

Indication : Créez une entité JPA `Departement` avec les champs `id` (Long) et `nom` (String). Un étudiant appartient à un seul département : ajoutez une relation `@ManyToOne` dans `Etudiant`. Mettez à jour les attributs d' `Etudiant` selon la spécification ci-dessous et générez le schéma via JPA (`ddl-auto=update`).

Attributs mis à jour de l'entité `Etudiant` :

Champ	Type
<code>id</code>	Long
<code>cin</code>	String
<code>nom</code>	String
<code>dateNaissance</code>	LocalDate
<code>email</code>	String
<code>anneePremiereInscription</code>	int
<code>departement</code>	Departement (ManyToOne)

Q8 — Mettre en place une architecture en couches propre

Indication : Refactorisez le projet en séparant clairement les responsabilités selon les packages suivants. Utilisez **Lombok** (`@Data` , `@Builder` , `@RequiredArgsConstructor`) et **JDK 25**. Un **DTO** est utilisé dans le Controller, un **Mapper** assure la conversion DTO ↔ entité, et le **Service** contient toute la logique métier.

Structure de packages attendue :

```
com.example.etudiants/  
├─ controller/  
├─ service/  
├─ repository/  
├─ entity/  
├─ dto/  
├─ mapper/  
└─ config/
```

Q9 — Ajouter une requête personnalisée `firstInscriptionYear` dans le repository

Indication : Dans l'interface `EtudiantRepository` (qui étend `JpaRepository`), ajoutez une méthode dérivée ou une requête JPQL permettant de récupérer les étudiants filtrés par leur `anneePremiereInscription`. Cette fonctionnalité expose un endpoint `GET /api/etudiants?annee=2022`.

```
List<Etudiant> findByAnneePremiereInscription(int annee);
```

Q10 — Ajouter les opérations CRUD complètes pour `Etudiant` et `Département`

Indication : Implémentez les endpoints REST standards (GET, POST, PUT, DELETE) pour les deux entités. Respectez les codes HTTP appropriés : `201 Created` pour la création, `204 No Content` pour la suppression, `404 Not Found` si la ressource est absente.

Méthode	URL	Description
GET	<code>/api/etudiants</code>	Liste tous les étudiants
GET	<code>/api/etudiants/{id}</code>	Récupère un étudiant
POST	<code>/api/etudiants</code>	Crée un étudiant
PUT	<code>/api/etudiants/{id}</code>	Met à jour un étudiant
DELETE	<code>/api/etudiants/{id}</code>	Supprime un étudiant
GET	<code>/api/departements</code>	Liste tous les départements
GET	<code>/api/departements/{id}</code>	Récupère un département
POST	<code>/api/departements</code>	Crée un département
PUT	<code>/api/departements/{id}</code>	Met à jour un département
DELETE	<code>/api/departements/{id}</code>	Supprime un département

Q11 — Ajouter la gestion des erreurs HTTP standard

Indication : Créez un `@RestControllerAdvice` global qui intercepte les exceptions et retourne des réponses JSON structurées avec le bon code HTTP. Gérez au minimum : `ResourceNotFoundException` (404), `MethodArgumentNotValidException` (400), et les erreurs génériques (500).


```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex)
    { ... }
}
```

Q12 — Ajouter la documentation Swagger / OpenAPI

Indication : Ajoutez la dépendance `springdoc-openapi-starter-webmvc-ui` dans votre `pom.xml`. Annotez vos controllers avec `@Operation` et `@ApiResponse` pour enrichir la documentation auto-générée. La documentation sera accessible à l'adresse `http://localhost:8080/swagger-ui.html`.

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.x.x</version>
</dependency>
```

Q13 — Ajouter Redis pour activer le cache

Indication : Ajoutez Redis comme couche de cache sur les endpoints de lecture. Utilisez `spring-boot-starter-data-redis` et activez le cache avec `@EnableCaching` dans votre classe de configuration. Annotez la méthode de service avec `@Cacheable` et invalidez le cache avec `@CacheEvict` sur les opérations d'écriture. Ajoutez un conteneur Redis dans votre `docker-compose.yml`.

```
@Cacheable(value = "etudiants")
public List<EtudiantDTO> findAll() { ... }

@CacheEvict(value = "etudiants", allEntries = true)
public EtudiantDTO save(EtudiantDTO dto) { ... }
```

Q14 — Créer un projet Jira Scrum avec deux sprints

Indication : Créez un projet de type **Scrum** sur Jira (version cloud gratuite). Définissez une **Epic** principale (ex. *"Gestion des Étudiants"*), décomposez-la en **User Stories** (ex. *"En tant qu'admin, je veux lister les étudiants"*), puis créez des **Tasks** techniques pour chaque story.

Organisez le backlog en deux sprints : **Sprint 1** couvrant la Partie 1, **Sprint 2** couvrant la Partie 2. Incluez une capture d'écran du board dans votre README.

Structure Jira attendue :

```
Epic : Gestion des Étudiants
├─ Sprint 1 – API REST de base (Partie 1)
│   ├── US-01 : Lister les étudiants
│   ├── US-02 : Dockeriser l'application
│   └── ...
└─ Sprint 2 – Enrichissement (Partie 2)
    ├── US-03 : Méthode age() + tests BDD
    ├── US-04 : Cache Redis
    └── ...
```

Structure du dépôt GitHub attendue

```
/projet-etudiants/
├─ api-spring-boot/
│   ├── src/
│   │   ├── main/
│   │   │   ├── java/com/example/etudiants/
│   │   │   │   ├── controller/
│   │   │   │   ├── service/
│   │   │   │   ├── repository/
│   │   │   │   ├── entity/
│   │   │   │   ├── dto/
│   │   │   │   ├── mapper/
│   │   │   │   └── config/
│   │   │   └── resources/
│   │   │       └── static/index.html
│   │   └── test/
│   │       └── resources/features/etudiant.feature
│   ├── Dockerfile
│   └── pom.xml
├─ k8s/
│   ├── etudiant-deployment.yaml
│   └── postgres-deployment.yaml
└─ docker-compose.yml
```

💡 **Rappel** : poussez régulièrement vos commits sur la branche `version-2` et liez chaque commit à un ticket Jira en incluant la clé du ticket dans le message de commit (ex. `git commit -m "PROJ-12 : ajout méthode age()"`).